



main(20261009-1713...
Code



train(4).txt
Document



dev(6).txt
Document



AI5707_Assignment_...
PDF

Explain the experiment from scratch , help me prepare for VIVA

Worked for 1m 8s

Analyzed

Analyzed

Analyzed

AI5707 Assignment 5 — Complete Viva Preparation

Experiment: Maximum Likelihood Parameter Estimation (MLE)

Let's prepare for your viva from absolute basics, as though you're explaining this experiment to your professor for the first time. We'll cover the mathematical intuition, every major part of your Python notebook, the graphs, and the questions a professor is likely to ask.

Your assignment has two parts: synthetic Gaussian data and a supplied real-world 2D dataset. Both use Gaussian probability models to estimate parameters and classify points into classes. The assignment requires decision-boundary plots, confusion matrices, and observations; your notebook also includes ROC curves and AUC as additional evaluation.

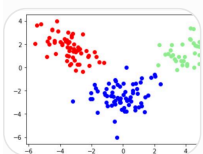
AI5707_Assignment_5(20261009-171321).pdf

1. First, what is the experiment actually about?

Imagine you are given thousands of points on a 2D graph. Each point belongs to one of three classes: Class 1, Class 2, or Class 3.

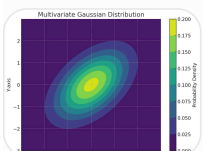
Your goal is to build a mathematical model that can answer:

- Given a new point (x, y) , which class does it belong to?
- How can we estimate each class's distribution from training data?
- How accurately can we classify points we have never used for training?



Step 1 — Understand the data

Each observation has two features, x and y , and a class label.



Step 2 — Estimate the distributions

Use training points to estimate the mean and covariance of each class through MLE.



Step 3 — Classify and evaluate

Predict the classes of held-out points and evaluate the predictions using accuracy, confusion matrices, decision boundaries, and ROC/AUC.

The entire experiment can be summarized in one sentence:

We estimate the parameters of Gaussian distributions from training data using maximum likelihood, then use those distributions to classify unseen data.

2. Prerequisites: understand these five concepts first

A. What is a feature vector?

A feature is a measurable property of an observation.

Your supplied dataset contains two features, x and y . We combine them into a vector:

$$\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$$

For example, the row

`</> Plain text`

`350, 1200, 1`

represents the feature vector $(350, 1200)^T$ with class label 1.

- x, y : input features.
- Class label: the correct category of that observation.
- The classifier uses the two features to predict the label.

B. What is a probability distribution?

A probability distribution describes how values are spread and how likely different values are.

For example, suppose the points belonging to Class 1 tend to appear near $(350, 1000)$. A probability distribution models that concentration and its spread.

A Gaussian distribution is especially useful because it is described by two parameters:

- **Mean:** the centre of the distribution.
- **Covariance:** the spread, orientation and relationship between features.

C. What is a mean?

The mean is the average position of the observations.

For n observations,

$$\hat{\boldsymbol{\mu}} = \frac{1}{n} \sum_{k=1}^n \mathbf{x}_k$$

Suppose the points along the x -axis are 2, 4, 6. Their mean is

$$\hat{\mu}_x = \frac{2 + 4 + 6}{3} = 4$$

For 2D data, the mean is a vector containing the average of each feature.

D. What is covariance?

Covariance tells us how features vary, including whether they tend to increase or decrease together.

For two features X and Y :

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

- Positive covariance: the features tend to move together.
- Negative covariance: one tends to increase as the other decreases.
- Zero covariance: no linear covariance is present.

Zero covariance does not, in general, prove statistical independence.

E. What is classification?

Classification means assigning an observation to a category.

For example, if a new point has coordinates (360, 1050), the model calculates how well it fits each class's Gaussian distribution and predicts the most probable class under its classification

rule.

This is a **supervised learning** experiment because the training data contains known class labels.

3. What is Maximum Likelihood Estimation (MLE)?

This is the **most important concept in your viva**.

MLE stands for Maximum Likelihood Estimation. It is a method for estimating unknown parameters of a probability distribution using observed data.

Imagine you have 1,000 points from a class, but you don't know the exact Gaussian distribution that generated them.

You want to estimate:

1. Where the distribution is centred.
2. How widely the points are spread.
3. Whether the spread is different along different directions.

MLE chooses parameter values that make the observed training data as likely as possible under the assumed probability model.

The mathematical idea

Let the observed data be

$$X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$$

and let θ represent the unknown parameters.

The likelihood function is

$$L(\theta) = \prod_{k=1}^n p(\mathbf{x}_k \mid \theta)$$

MLE estimates the parameters by maximizing this likelihood:

$$\hat{\theta}_{MLE} = \arg \max_{\theta} L(\theta)$$

In words: find the parameters that make the observed samples most likely.

Because multiplying thousands of small probabilities can cause numerical underflow, we usually maximize the log-likelihood instead:

$$\ell(\theta) = \sum_{k=1}^n \log p(\mathbf{x}_k \mid \theta)$$

The logarithm is a strictly increasing function, so maximizing likelihood and maximizing log-likelihood give the same parameter estimate.

Why does your experiment use MLE?

The assignment specifies that we should estimate the parameters of three Gaussian classes from the training data. MLE provides estimates for the mean vector and covariance matrix of each class.

For class i , the notebook calculates:

$$\hat{\boldsymbol{\mu}}_i = \frac{1}{n_i} \sum_{k=1}^{n_i} \mathbf{x}_k$$

and

$$\hat{\boldsymbol{\Sigma}}_i = \frac{1}{n_i} \sum_{k=1}^{n_i} (\mathbf{x}_k - \hat{\boldsymbol{\mu}}_i)(\mathbf{x}_k - \hat{\boldsymbol{\mu}}_i)^T$$

Here:

- n_i is the number of training samples belonging to class i .
- $\hat{\boldsymbol{\mu}}_i$ is the estimated mean.
- $\hat{\boldsymbol{\Sigma}}_i$ is the estimated covariance matrix.
- T denotes transpose.

Viva trap: The covariance denominator is n_i , not $n_i - 1$, because the notebook uses the maximum-likelihood estimate. Dividing by $n_i - 1$ is the usual unbiased sample covariance estimator instead.

4. What is a Gaussian distribution?

Your experiment assumes that the observations within each class can be modelled using a multivariate Gaussian, also called a multivariate normal distribution.

For a d -dimensional feature vector $\mathbf{x}$, its probability density function (PDF) is

$$p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}|^{1/2}} \times \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right]$$

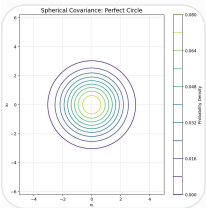
Don't panic about the equation. You need to understand what each component means.

Symbol	Meaning
$\mathbf{x}$	Feature vector being evaluated
$\boldsymbol{\mu}$	Mean vector; the centre of the distribution
$\boldsymbol{\Sigma}$	Covariance matrix; spread and orientation
$ \boldsymbol{\Sigma} $	Determinant of the covariance matrix
$\boldsymbol{\Sigma}^{-1}$	Inverse covariance matrix
d	Number of features; here $d = 2$
$p(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$	Probability density at the given point

The exponential term measures how far the point is from the mean, taking the covariance structure into account. The other terms normalize the density.

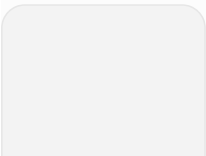
A Gaussian density is not necessarily a probability between 0 and 1 at a particular point. It is a *density*; probabilities are obtained by integrating over regions.

What does the Gaussian look like in 2D?



Isotropic Gaussian

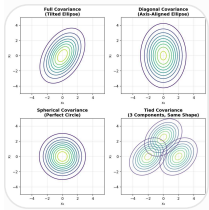
Equal spread in every direction. Its contours are circles.



Diagonal covariance Gaussian



The two features may have different variances, but the covariance matrix assumes zero cross-covariance. Its contours are axis-aligned ellipses.



Full covariance Gaussian

Can represent different variances and nonzero covariance. Its contours can be tilted ellipses.

5. The three covariance models in your experiment

This is another high-priority viva topic. Your notebook compares three covariance assumptions: isotropic, diagonal and full.

For 2D data, the covariance matrix is

$$\Sigma = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix}$$

The diagonal entries represent the feature variances. The off-diagonal entries represent covariance between the features.

1. Isotropic covariance

$$\Sigma = \sigma^2 I = \begin{bmatrix} \sigma^2 & 0 \\ 0 & \sigma^2 \end{bmatrix}$$

Both features have equal variance, and their covariance is zero. The Gaussian contours are circular.

2. Diagonal covariance

$$\Sigma = \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_y^2 \end{bmatrix}$$

Each feature can have a different variance, but cross-covariance is assumed to be zero. The contours are axis-aligned ellipses.

3. Full covariance

$$\Sigma = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix}$$

The model estimates both variances and the cross-covariance. It can represent tilted ellipses.

How many parameters are estimated?

For a 2D Gaussian, the mean always has two parameters.

Covariance model	Mean parameters	Covariance parameters	Total per class
Isotropic	2	1	3
Diagonal	2	2	4
Full	2	3	5

For a full covariance matrix, the two off-diagonal elements must be equal, so there are three distinct covariance parameters, not four.

Viva question: Which covariance model is most flexible?

Answer: The full covariance model, because it estimates both feature variances and their cross-covariance. However, more flexibility does not guarantee better test performance. It can be less reliable with limited training data, and its performance depends on whether the Gaussian assumption is appropriate.

What does shared versus class-specific covariance mean?

Your notebook also varies whether classes share their covariance matrix.

- **Shared covariance:** all three classes use the same covariance matrix, although their means can differ.
- **Class-specific covariance:** each class has its own covariance matrix.

For example, shared covariance means

$$\Sigma_1 = \Sigma_2 = \Sigma_3$$

while class-specific covariance permits

$$\Sigma_1 \neq \Sigma_2 \neq \Sigma_3$$

This is a separate modelling choice from whether covariance is isotropic, diagonal or full.

Your synthetic experiment combines three data-generating covariance types with two covariance-sharing settings. It then evaluates the three fitted covariance models for each setting.

6. How does the Gaussian classifier predict a class?

Estimating the distributions is only half the experiment. We must now use them to classify unseen points.

For a new point $\mathbf{x}$, the classifier calculates a score for every class.

The Bayesian classification rule is

$$\hat{C}(\mathbf{x}) = \arg \max_i p(\mathbf{x} \mid C_i) P(C_i)$$

where:

- $p(\mathbf{x} \mid C_i)$ is the likelihood of the point under class i 's Gaussian.
- $P(C_i)$ is the prior probability of class i .
- $\arg \max$ chooses the class with the highest score.

Your notebook estimates the class priors from the training data:

$$\hat{P}(C_i) = \frac{n_i}{N}$$

Here, n_i is the number of training samples in class i , and N is the total number of training samples.

If the classes are equally represented, their estimated priors will be equal.

Why does the code use log-likelihood?

The notebook calculates the Gaussian log-likelihood and adds the log-prior:

$$g_i(\mathbf{x}) = -\frac{1}{2} \left[d \log(2\pi) + \log |\Sigma_i| + (\mathbf{x} - \mu_i)^T \Sigma_i^{-1} (\mathbf{x} - \mu_i) \right] + \log P(C_i)$$

It then chooses

$$\hat{C}(\mathbf{x}) = \arg \max_i g_i(\mathbf{x})$$

This is mathematically equivalent to maximizing the likelihood multiplied by the prior, because the logarithm preserves the ordering of positive values.

Walk through a simple example

Suppose a new point produces the following hypothetical scores:

Class	Log-score
Class 1	-12.4
Class 2	-4.2
Class 3	-8.7

The classifier predicts **Class 2**, because -4.2 is the largest score.

Notice that the highest score is the least negative one.

What does `np.argmax()` do?

In your code, `np.argmax(scores, axis=1)` returns the index of the largest class score for each observation. That index is mapped back to the actual class label.

What is numerical regularization?

The notebook adds a very small value to the diagonal of each fitted covariance matrix:

$$\Sigma_{\text{stable}} = \Sigma + \lambda I$$

where $\lambda = 10^{-6}$ in your implementation.

This helps prevent numerical problems when inverting a covariance matrix or calculating its determinant. It does not replace MLE; it is a small numerical-stability adjustment.

7. Experiment Part 1 — Synthetic Gaussian data

Let's connect the theory to your actual notebook.

Your implementation uses three classes, with these predefined mean vectors:

$$\begin{aligned}\mu_1 &= (-4, -1.5)^T \\ \mu_2 &= (0, 4)^T \\ \mu_3 &= (4, -1)^T\end{aligned}$$

It generates **10,000 observations per class**, giving 30,000 observations per generated dataset.

What happens in the code?

1. Define three class means.
2. Generate samples using multivariate Gaussian distributions with the selected covariance structures.
3. Vary whether the generating covariance is shared across classes or class-specific.
4. Split each dataset into 80% training and 20% testing, preserving class proportions.
5. Fit isotropic, diagonal and full Gaussian models using only training data.
6. Predict the test labels and compute the confusion matrix and accuracy.
7. Plot the decision boundaries and the held-out test points.

The notebook explores three generating covariance types and two sharing settings, then evaluates three fitted covariance types in each scenario. That gives $3 \times 2 \times 3 = 18$ model evaluations.

Why use synthetic data?

Because the generating distributions are controlled, we can investigate whether different covariance assumptions affect classification performance. We know how the data was generated and can compare the fitted model assumptions with those conditions.

Why use 80:20 splitting?

For a dataset of 30,000 samples:

$$N_{\text{train}} = 0.8(30000) = 24000$$

$$N_{\text{test}} = 0.2(30000) = 6000$$

That means 24,000 observations are used for estimating parameters and 6,000 are reserved for evaluation.

Important distinction: The synthetic experiment uses a stratified 80:20 split. For the supplied real-data experiment, the notebook uses the separate training and development files provided to you.

8. Experiment Part 2 — Your supplied 2D dataset

The supplied data contains three columns:

</> Plain text

```
x, y, class_label
```

For this part, the notebook:

1. Loads the training and held-out datasets.
2. Checks that the feature values are numeric and finite.
3. Checks that class labels are valid integers and that both files contain the same set of classes.
4. Estimates class means and covariance matrices from training data.
5. Fits isotropic, diagonal and full Gaussian classifiers.
6. Evaluates their predictions on the held-out dataset.
7. Generates confusion matrices, classification reports, decision-boundary plots and ROC/AUC results.

The real-data section is especially useful for viva questions because it demonstrates that MLE can be applied to observations supplied by a dataset, rather than only to data generated from known distributions.

One practical detail to fix before your viva

Your uploaded files are named `train(4).txt` and `dev(6).txt`, whereas the notebook's file-loading cell initially searches for `train.txt` and `dev.txt`.

If you run the notebook in a folder containing only the uploaded filenames, that cell may report that real-data evaluation is pending. Either rename the files to match the expected names or update the candidate filenames in the notebook. Then run the notebook and inspect the actual metrics before memorizing any numerical results.

Also, the assignment describes an 80:20 split, so check the printed row counts and proportions. A pre-supplied train/dev split should not automatically be assumed to have exactly that ratio.

9. Confusion matrix — a viva favourite

A confusion matrix compares the actual labels against the predicted labels.

For three classes, it is a 3×3 matrix:

$$CM = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

In your notebook:

- Rows represent **true classes**.
- Columns represent **predicted classes**.
- Diagonal entries are correct predictions.
- Off-diagonal entries are misclassifications.

For example, suppose a confusion matrix is

$$CM = \begin{bmatrix} 90 & 5 & 5 \\ 3 & 92 & 5 \\ 4 & 6 & 90 \end{bmatrix}$$

These are illustrative values, not your experiment's results.

The first row means that among 100 actual Class 1 samples, 90 were correctly classified as Class 1, five were predicted as Class 2, and five as Class 3.

Accuracy

Accuracy measures the fraction of all predictions that are correct.

$$\text{Accuracy} = \frac{\sum_i CM_{ii}}{\sum_i \sum_j CM_{ij}}$$

For the illustrative matrix above,

$$\text{Accuracy} = \frac{90 + 92 + 90}{300} = 90.67\%$$

Precision, recall and F1-score

These metrics tell us more than accuracy alone.

Metric	Formula	Interpretation
Precision	$TP / (TP + FP)$	Of the samples predicted positive for a class, how many were correct?
Recall	$TP / (TP + FN)$	Of the actual samples of a class, how many were correctly found?
F1-score	$2PR / (P + R)$	Harmonic mean of precision and recall

For multiclass classification, these metrics can be computed for each class by treating that class as positive and the remaining classes as negative.

Your notebook uses a classification report to display class-wise precision, recall, F1-score and support.

Viva question: Why isn't accuracy always sufficient?

If one class dominates the dataset, a model can obtain high accuracy while performing poorly on a minority class. A confusion matrix and class-wise recall reveal this problem more clearly.

10. Decision boundaries — what should you say when the professor points at a plot?

A decision boundary is the set of points where two or more classes have equal classification scores.

For classes i and j , the boundary satisfies

$$g_i(\mathbf{x}) = g_j(\mathbf{x})$$

On one side, one class has the highest score; on the other side, a different class wins.

In the notebook, the plotting function evaluates the classifier over a grid of (x, y) coordinates. It colours each grid location according to the predicted class and overlays the held-out test points.

Why do the boundaries have different shapes?

The covariance assumptions affect the Gaussian score functions.

- With a shared covariance matrix across classes, the Gaussian quadratic terms cancel when comparing class scores. The resulting boundaries are linear, assuming the same Gaussian classifier and class-prior treatment used here.

- With class-specific full covariance matrices, the quadratic terms generally do not cancel, so the boundaries can be curved.
- With isotropic or diagonal class-specific covariance matrices, the boundaries can also be curved if the classes have different covariance values.

What can we learn from the plot?

- Clear class separation suggests that the fitted distributions distinguish the classes well.
- Overlapping regions indicate that observations may be difficult to distinguish.
- Test points on the wrong side of a boundary indicate classification errors.
- A complex boundary is not automatically better; test performance determines whether the model generalizes well.

11. ROC curve and AUC — the additional part of your notebook

Your assignment does not explicitly require ROC/AUC, but your notebook includes this additional evaluation.

ROC stands for **Receiver Operating Characteristic**.

It plots the true positive rate against the false positive rate at different decision thresholds.

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

- **TPR**: the fraction of actual positive samples correctly identified.
- **FPR**: the fraction of actual negative samples incorrectly identified as positive.

For multiclass classification, your notebook uses a one-vs-rest approach. It evaluates each class against all the remaining classes.

What is AUC?

AUC stands for Area Under the ROC Curve.



AUC close to 1

The model generally ranks positive examples above negative examples very well.

– AUC around 0.5

Discrimination is approximately at the level of random ranking in a standard binary setting.

↘ AUC below 0.5

The ranking is worse than random for the chosen positive-class orientation; reversing the scores would reverse the ranking.

Your notebook calculates per-class AUC and macro- and micro-average AUC.

- **Macro AUC:** the arithmetic mean of the available per-class AUC values, giving each class equal weight.
- **Micro AUC:** pools the one-vs-rest decisions across classes before calculating the curve, so each class/sample decision contributes to the overall calculation.

The ROC curves use normalized class posterior probabilities obtained by applying softmax to the Gaussian log-scores.

Important: AUC measures ranking/discrimination across thresholds; accuracy measures correctness at the classifier's chosen decision rule. A model can have good AUC without having the best accuracy at that particular rule.

12. Understand the notebook's main code functions

These are the functions you should be able to explain without reading every line of code.

Function	Purpose
<code>covariance_for_generation()</code>	Constructs the synthetic covariance matrix for the selected type.
<code>generate_synthetic_dataset()</code>	Generates samples for the three synthetic Gaussian classes.
<code>fit_gaussian_classifier()</code>	Estimates means, covariances and class priors from labelled training data.
<code>gaussian_class_scores()</code>	Calculates a Gaussian log-score plus log-prior for every class.
<code>predict_gaussian()</code>	Chooses the class with the highest score.
<code>evaluate_model()</code>	Produces predictions, accuracy, confusion matrix and classification report.
<code>plot_decision_boundary()</code>	Displays the model's predicted regions with test samples.
<code>load_dataset()</code>	Reads and validates the supplied 2D dataset.

`plot_multiclass_roc()`

Plots one-vs-rest ROC curves and calculates AUC metrics.

Explain `fit_gaussian_classifier()` in particular

This function is the heart of the experiment.

1. Convert features to a numeric array and validate the inputs.
2. Identify the unique class labels.
3. For each class, calculate its mean.
4. Centre its observations by subtracting that mean.
5. Calculate the covariance using the MLE denominator n_i .
6. Estimate class priors from training class counts.
7. Apply the requested covariance structure: isotropic, diagonal or full.
8. If shared covariance is selected, calculate a pooled covariance and assign it to all classes.
9. Add the small diagonal regularizer for numerical stability.
10. Return the fitted model parameters.

Then the classifier uses those parameters to score and predict new samples.

13. Rapid-fire viva questions with model answers

Read each question and try to answer it aloud before revealing the suggested answer.

Q1. What is the main objective of this experiment?



Q2. What does MLE stand for?



Q3. Why do we use a Gaussian distribution?



Q4. Why divide the covariance estimate by n rather than $n - 1$?



Q5. What is the difference between covariance and variance?



Q6. What is the difference between isotropic, diagonal and full covariance?



Q7. What is the difference between shared and class-specific covariance?



Q8. Why is this supervised learning?



Q9. Why do we keep a test set?



Q10. What does a confusion matrix tell us?



Q11. What is a decision boundary?



Q12. Why use log-likelihood?



Q13. What is a class prior?



Q14. Why add a small value to the covariance diagonal?



Q15. What is the difference between accuracy and AUC?



Q16. Why might full covariance perform worse than diagonal covariance?



Q17. Does high training accuracy guarantee high test accuracy?



Q18. What does a high off-diagonal covariance value indicate?



14. Your 60-second viva introduction

Practise saying this naturally, without trying to memorize every mathematical expression.

Writing



My experiment is based on Maximum Likelihood Parameter Estimation for Gaussian classification, as part of the Pattern Recognition and Machine Learning laboratory.

The main objective is to estimate the parameters of different Gaussian classes from training data and

use these estimates to classify unseen observations.

The experiment has two parts. First, I generate synthetic data for three Gaussian classes, using isotropic, diagonal and full covariance structures, with both shared and class-specific covariance settings. Second, I apply the same method to the supplied two-dimensional dataset.

For each class, I estimate the mean vector and covariance matrix using maximum likelihood estimation. I then calculate Gaussian log-likelihood scores, incorporate the class priors, and assign each test sample to the class with the highest score.

Finally, I evaluate the classifier using accuracy, confusion matrices, classification reports and decision-boundary plots. I also include ROC curves and AUC as additional evaluation metrics.

The experiment demonstrates how covariance assumptions affect the decision boundaries and classification performance of Gaussian classifiers.

15. Final revision checklist

Before entering the viva, make sure you can explain each item without looking at your notes.

Viva readiness

0/12

- ☐ Define MLE and explain why it is used.
- ☐ Write the MLE formulas for the mean and covariance.
- ☐ Explain every term in the multivariate Gaussian PDF.
- ☐ Distinguish isotropic, diagonal and full covariance.
- ☐ Explain shared versus class-specific covariance.
- ☐ Describe the training, prediction and evaluation pipeline.
- ☐ Interpret a confusion matrix and calculate accuracy.
- ☐ Explain what decision boundaries represent.
- ☐ Explain ROC, AUC, precision, recall and F1-score.
- ☐ Identify the purpose of the main Python functions.
- ☐ Know the actual test accuracy and confusion matrices from your executed notebook.

☐ Confirm that the real-data evaluation completed successfully.

My strongest advice: **prioritize MLE, covariance matrices, Gaussian classification, and the confusion matrix first**. These concepts form the core of your experiment. Once you understand them, the plots and Python functions become much easier to explain.

And remember, you don't need to recite the entire code. You need to explain what each stage does, why it is necessary, and how the result supports the experiment's objective.

</>

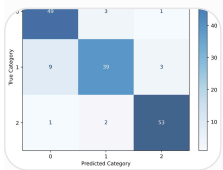
explain the plots, and why are there different plots

Worked for 15s

16. Why are there so many different plots in your experiment?

The key idea is that **each plot answers a different question**. You are not repeating the same experiment unnecessarily; you are examining the classifier from different perspectives.

Your notebook produces three main kinds of visualizations:



Plot 1: Confusion matrix

Question answered: *Which classes does the model classify correctly, and which classes does it confuse?*

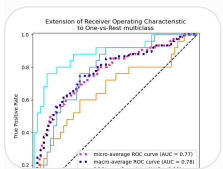
It shows counts of correct and incorrect predictions.



Plot 2: Decision-boundary plot

Question answered: *How does the classifier divide the feature space into classes?*

It shows predicted class regions and the held-out test observations.



Plot 3: ROC curve

Question answered: *How well can the model distinguish one class from the others at different decision thresholds?*

It shows true-positive rate against false-positive rate.

The first two are explicitly required by the assignment. The ROC plot is an additional analysis in your notebook. [AI5707_Assignment_5\(20261009-171321\).pdf](#)

17. Why does your notebook generate so many decision-boundary plots?

This is probably the most important thing to understand for your viva.

Your synthetic experiment changes **two things independently**:

- 1. How the data was generated.
- 2. Which covariance model the classifier uses.

You have three data-generating types, two covariance-sharing settings, and three fitted models.

$3 \times 2 \times 3 =$

18 plots

Here is the complete breakdown.

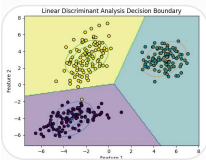
What changes?	Options	Number
Generating covariance	Isotropic, diagonal, full	3
Generating covariance sharing	Shared, class-specific	2
Fitted covariance model	Isotropic, diagonal, full	3
Total synthetic decision-boundary plots		18

Why do this? Because the data-generating process and the model's assumptions do not always match.

For example, suppose the data was generated with full covariance, but you fit an isotropic Gaussian classifier. The classifier assumes equal variance in every direction, even though the actual data may have tilted elliptical distributions.

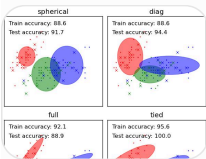
Comparing the resulting plots shows how that mismatch affects classification.

What do the different shapes mean?



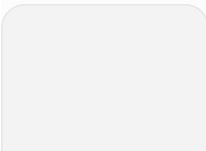
Case A: Isotropic model

The model assumes circular Gaussian distributions. If the covariance is shared between classes, the boundaries are straight lines. With different class-specific isotropic variances, boundaries can be curved.



Case B: Diagonal model

The model permits different spreads along the *x* and *y* axes. It cannot represent a nonzero covariance between these features, so its estimated shapes remain axis-aligned.



Case C: Full covariance model

The model can capture feature correlations and tilted ellipses. With different covariance matrices for different classes, the resulting boundaries may be curved.

These are illustrative examples of the types of plots, not screenshots of your actual notebook outputs.

What to say in the viva: "I generate multiple plots to compare how covariance assumptions affect the decision regions and classification performance. The generating covariance describes the actual synthetic data, whereas the fitted covariance describes the assumptions used by the classifier."

18. Why are there different plots for the real dataset?

Your real-data experiment fits three different classifiers to the same training data:

- Isotropic covariance
- Diagonal covariance
- Full covariance

For each model, your notebook produces a confusion-matrix heatmap and a decision-boundary plot. It also generates a separate ROC plot for each model.

Your real-data visualizations

3 Decision-boundary plots

Compare how the three covariance models divide the same feature space.

3 Confusion-matrix heatmaps

Compare the errors made by each covariance model.

3 ROC plots

Compare how well each model ranks the classes at different thresholds.

That is 9 real-data plots, in addition to the 18 synthetic decision-boundary plots. The synthetic confusion matrices are printed as tables rather than displayed as heatmaps.

19. How to interpret all the plots together

Imagine the full covariance model has a higher test accuracy than the isotropic model.

You should investigate three things:

1. **Decision boundary:** Does the full covariance model better match the shapes and overlap of the observed classes?

2. **Confusion matrix:** Does it reduce errors for particular classes, or does it only improve the overall count?
3. **ROC curve:** Does it distinguish the classes better across different thresholds, not just at the default classification rule?

If all three indicate improvement, that provides stronger evidence that the model is performing better. However, the results need not agree perfectly: accuracy, class-specific errors and AUC measure different aspects of performance.

One final distinction: the decision-boundary plot colours each region according to the class predicted by the model. It does **not** directly show the probability that the prediction is correct. The confusion matrix and ROC curve provide complementary information about predictive performance.

20. Quick viva test

1. Why are there 18 synthetic decision-boundary plots?

- ☐ Because the notebook uses 18 classes
- ☐ Because 3 generating covariance types $\times$ 2 sharing settings $\times$ 3 fitted models
- ☐ Because every test point gets a separate plot

2. What does a confusion matrix show that a decision-boundary plot does not show directly?

- ☐ The exact counts of correct and incorrect predictions by class
- ☐ The Gaussian mean vectors
- ☐ The shape of the covariance matrix

3. Why is ROC included?

- ☐ It estimates the Gaussian mean
- ☐ It replaces the confusion matrix
- ☐ It evaluates discrimination across different thresholds

4. If a decision boundary is curved, what can that suggest?

- ☐ The classifier must be incorrect

- ☐ The class score functions may contain different quadratic terms
- ☐ The model has 100% accuracy

Try again

Remember this for your viva: different plots are not just for presentation. Each one helps you test a different part of the model: the geometry of classification, the types of errors, or the ability to distinguish classes across thresholds.

explain why there are 18 plots and why are there variations

The **18 plots** in your assignment are generated because the notebook compares different combinations of **data-generation assumptions**, **covariance-sharing assumptions**, and **fitted Gaussian models**. This is an important point to explain in your viva.

1. Why exactly 18 plots?

Your notebook varies three things:

3 Data-generation covariance types

- Isotropic
- Diagonal
- Full

2 Covariance-sharing settings

- Shared covariance across classes
- Different covariance for each class

3 Fitted Gaussian classifier types

- Isotropic
- Diagonal
- Full

Total decision-boundary plots

$$3 \times 2 \times 3 = 18$$

The notebook generates a decision-boundary plot for every combination.

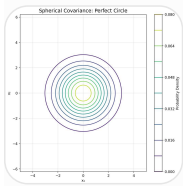
For example, when the data are generated using an isotropic covariance structure with shared covariance, the notebook tests three fitted models: isotropic, diagonal, and full. That gives three plots for that particular scenario.

The same process is repeated for all six data-generation scenarios.

2. What causes the variations in the plots?

There are two main sources of variation.

A. The generated data are different

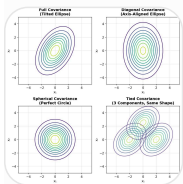


1. Isotropic covariance

The variance is equal in every direction. The Gaussian distribution has circular contours in 2D.

$$\Sigma = \begin{bmatrix} \sigma^2 & 0 \\ 0 & \sigma^2 \end{bmatrix}$$

Expected shape: circular

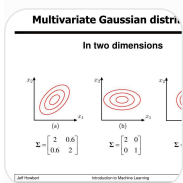


2. Diagonal covariance

The two features can have different variances, but their covariance is zero.

$$\Sigma = \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_y^2 \end{bmatrix}$$

Expected shape: axis-aligned ellipse, or circle if variances match



3. Full covariance

The model allows different variances and nonzero covariance between features.

$$\Sigma = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix}$$

Expected shape: potentially tilted ellipse

Because the data are generated under different assumptions, the positions and shapes of the class clusters can change.

B. The fitted classifier makes different assumptions

Even if the same dataset is used, the three fitted models may learn different decision boundaries.

Fitted model	What it assumes	Possible boundary
Isotropic	Equal variance in all directions	Often simple and restrictive
Diagonal	Different feature variances, no cross-covariance	More flexible
Full	Different variances and feature correlation	Most flexible of the three

A **shared covariance** means all classes use one common covariance matrix. Under Gaussian classification with shared covariance, the decision boundaries are linear.

With **class-specific covariance**, each class can have its own covariance matrix. This can produce curved, quadratic decision boundaries.

The key idea is that a model's assumptions affect how well it separates the classes.

3. How the 18 plots are organized

Each row below represents a data-generation scenario. Every scenario produces three plots, one for each fitted covariance model.

Data-generation type	Covariance setting	Fitted models	Plots
Isotropic	Shared	Isotropic, diagonal, full	3
Isotropic	Class-specific	Isotropic, diagonal, full	3
Diagonal	Shared	Isotropic, diagonal, full	3
Diagonal	Class-specific	Isotropic, diagonal, full	3
Full	Shared	Isotropic, diagonal, full	3
Full	Class-specific	Isotropic, diagonal, full	3
Total	6 scenarios	3 models per scenario	18

One important implementation detail: in your notebook, the `shared_covariance` flag for the fitted models is taken from the outer scenario loop. So the three fitted types in a given scenario share that same setting, while differing in their covariance type.

4. What should you say in the viva?

Viva-ready answer

“We generate 18 decision-boundary plots because we experiment with three covariance structures for generating the data, two covariance-sharing settings, and three covariance

types for the fitted Gaussian classifiers. Therefore, the total is $3 \times 2 \times 3 = 18$.

The plots vary because the covariance assumptions change the estimated distributions and how the classifier separates the classes. Isotropic covariance assumes equal variance in all directions, diagonal covariance allows different variances for each feature, and full covariance also models correlations between features. Comparing these plots helps us understand how the choice of covariance model affects classification and decision boundaries.”

Remember: the 18 plots are the *synthetic-data decision-boundary plots*. They are not 18 different kinds of graphs; they are the same kind of graph repeated under different experimental conditions.

what is fitted model mean?

Great question! This is one of the most important concepts in your MLE assignment. Let's understand it with a simple example.

1. What is a fitted model?

A **fitted model** is a model whose parameters have been learned from training data.

In your assignment, you use a Gaussian classifier. Before fitting, the classifier doesn't know the mean, covariance, or prior probability of each class. It estimates these values from the training samples.

Think of it like this:

Training data

Feature values and their class labels



Fit the Gaussian model using MLE

Estimate means, covariance matrices and class priors



Fitted model

The learned parameters are now available for prediction

2. A simple example

Suppose your training dataset contains two classes of students based on two features: mathematics marks and programming marks.

- Class A: students with lower marks on average.
- Class B: students with higher marks on average.

Before training, the Gaussian classifier doesn't know the distribution of either class.

After fitting, it might estimate:

Parameter	Class A	Class B
Mean mathematics mark	40	80
Mean programming mark	45	85

These numbers are just illustrative. The model also estimates covariance matrices and class prior probabilities.

Once the parameters have been estimated, the model is **fitted**. Given a new student's marks, it can predict whether the student belongs to Class A or Class B.

3. What does “fitted model” mean in your code?

Your notebook calls:

</> Python

```
model = fit_gaussian_classifier(  
    X_train,  
    y_train,  
    covariance_type="full"  
)
```

Here's what happens:

1. `X_train` supplies the training features.
2. `y_train` supplies the correct class labels.
3. `covariance_type="full"` tells the function which covariance structure to estimate.
4. The function estimates the Gaussian parameters for each class.
5. `model` stores those estimated parameters.

Then you use the fitted model to predict:

</> Python

```
y_pred = predict_gaussian(model, X_test)
```

Notice the difference:

- **Fitting:** learn the parameters from training data.
- **Prediction:** use the learned parameters to classify unseen data.

4. Why fit three different models in your assignment?

You fit three versions of the classifier to compare their assumptions.

- **Isotropic model:** assumes equal variance in every direction.
- **Diagonal model:** permits different feature variances but assumes zero covariance between features.
- **Full model:** also estimates covariance between features.

The same training data can therefore produce three fitted models with different parameter estimates and potentially different decision boundaries.

Viva one-liner: "A fitted model is a statistical model whose unknown parameters have been estimated from training data using maximum likelihood estimation. In our assignment, the fitted Gaussian classifier uses the estimated class means, covariance matrices, and priors to classify test samples."